

Tecnología Digital VI

Arquitecturas famosas y Transfer Learning

Licenciatura en Tecnología Digital - UTDT

Dando el salto

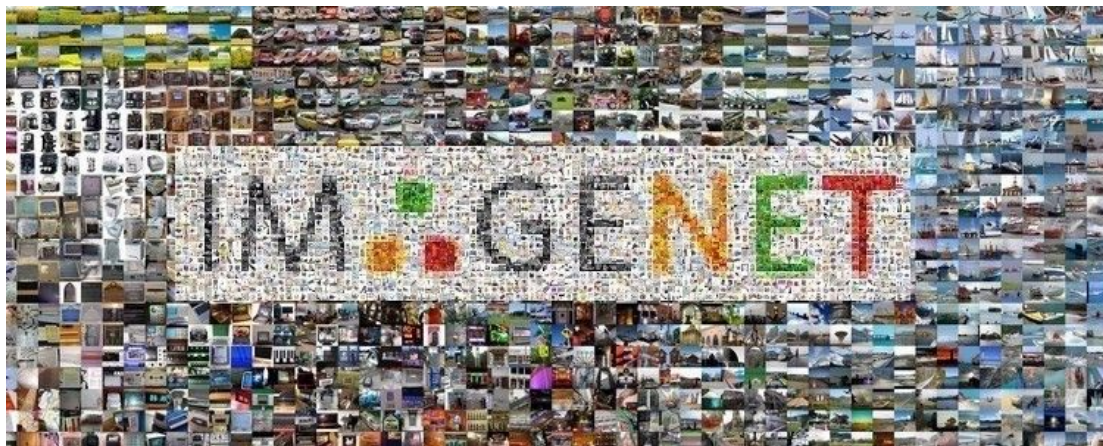
Hasta ahora venimos haciendo *redes a mano*. Eso implica que cada nuevo problema que tengamos tenemos que definir varias cuestiones (arquitectura, loss function, hiperparámetros, entrenamiento, etc) y siempre entrenar nuestra red desde cero.

Vamos a ver redes neuronales famosas que fueron muy útiles para resolver ciertos problemas, pero que además nos sirven al reutilizarlas en otros contextos.

ImageNet

ImageNet es posiblemente el dataset de imágenes más importante en Inteligencia Artificial.

Se creó unos años antes del 2010 con la idea tener un muy extenso dataset de imágenes *naturales* con el que poder trabajar ciertos problemas como clasificación de objetos.



ImageNet Challenge (ILSVRC)

El dataset tiene varias versiones pero originalmente constaba de unas 14 millones de imágenes, divididas en casi 22k clases. Otra versión bastante utilizada es el ImageNet-1K que tiene “solo” 1000 clases y un poco más de un millón de imágenes.

El dataset se hizo muy conocido al comenzar a realizarse una competencia anual en la que los mejores investigadores de Computer Vision del mundo enviaban sus modelos para competir.

ImageNet Challenge

Las primeras 2 competencias ([2010](#) y [2011](#)) tuvieron unos pocos equipos de diferentes universidades y centros de investigación.

En ambas competencias el ganador fue un modelo clásico (SVM) trabajando no sobre los píxeles directamente sino sobre features extraídas de la imágenes con algoritmos específicos como HOG.

En [2012](#) AlexNet, una CNN, entró a la competencia y la ganó (por goleada). Fue el principio del fin del Machine Learning Clásico en los problemas de imágenes*.

No fue la primera Red Neuronal Convolutacional conocida. Yann LeCun con sus LeNet y otros investigadores ya venían trabajando varios años antes. Sin embargo, se considera la primera red neuronal convolutacional profunda con aplicación en un problema *real*.

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet ILSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called “dropout” that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

AlexNet – Arquitectura

3.5 Overall Architecture

Now we are ready to describe the overall architecture of our CNN. As depicted in Figure 2, the net contains eight layers with weights; the first five are convolutional and the remaining three are fully-connected. The output of the last fully-connected layer is fed to a 1000-way softmax which produces a distribution over the 1000 class labels. Our network maximizes the multinomial logistic regression objective, which is equivalent to maximizing the average across training cases of the log-probability of the correct label under the prediction distribution.

The kernels of the second, fourth, and fifth convolutional layers are connected only to those kernel maps in the previous layer which reside on the same GPU (see Figure 2). The kernels of the third convolutional layer are connected to all kernel maps in the second layer. The neurons in the fully-connected layers are connected to all neurons in the previous layer. Response-normalization layers follow the first and second convolutional layers. Max-pooling layers, of the kind described in Section 3.4, follow both response-normalization layers as well as the fifth convolutional layer. The ReLU non-linearity is applied to the output of every convolutional and fully-connected layer.

The first convolutional layer filters the $224 \times 224 \times 3$ input image with 96 kernels of size $11 \times 11 \times 3$ with a stride of 4 pixels (this is the distance between the receptive field centers of neighboring neurons in a kernel map). The second convolutional layer takes as input the (response-normalized and pooled) output of the first convolutional layer and filters it with 256 kernels of size $5 \times 5 \times 48$. The third, fourth, and fifth convolutional layers are connected to one another without any intervening pooling or normalization layers. The third convolutional layer has 384 kernels of size $3 \times 3 \times 256$ connected to the (normalized, pooled) outputs of the second convolutional layer. The fourth convolutional layer has 384 kernels of size $3 \times 3 \times 192$, and the fifth convolutional layer has 256 kernels of size $3 \times 3 \times 192$. The fully-connected layers have 4096 neurons each.

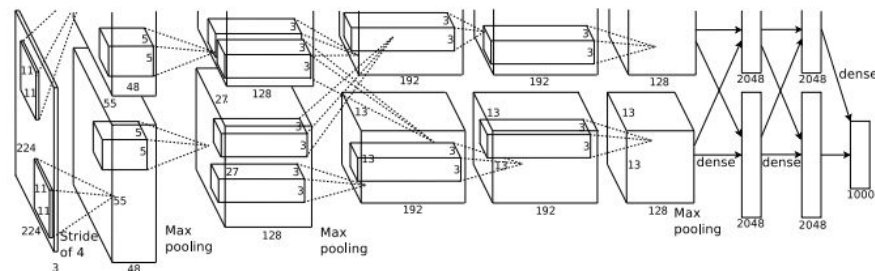


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

AlexNet – Arquitectura

La arquitectura en sí ya era un breakthrough. Si bien no había nada *desconocido* en las diferentes capas utilizadas, 8 capas de (muchos) parámetros a aprender era considerado algo MUY profundo.

Solo con estas capas había 60 millones de parámetros para entrenar.

[227x227x3] INPUT
[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0
[27x27x96] **MAX POOL1**: 3x3 filters at stride 2
[27x27x96] **NORM1**: Normalization layer
[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2
[13x13x256] **MAX POOL2**: 3x3 filters at stride 2
[13x13x256] **NORM2**: Normalization layer
[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1
[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1
[6x6x256] **MAX POOL3**: 3x3 filters at stride 2
[4096] **FC6**: 4096 neurons
[4096] **FC7**: 4096 neurons
[1000] **FC8**: 1000 neurons (class scores)

AlexNet – Aportes

Varias cuestiones particulares de la arquitectura y entrenamiento:

- Entrenamiento en más de una GPU.
- Capas de normalización.
- Data Augmentation:
 - Traslaciones y reflexiones.
 - PCA sobre el conjunto de píxeles RGB para intensificar las imágenes de manera *natural*.
- Dropout en las dos primeras capas fully-connected para reducir fuerte el overfitting.
- SGD, batch size de 128, momentum 0.9. LR reducido en plateaus.
- Ensamble de CNNs.

AlexNet – Aportes

Más allá de los resultados cuantitativos muy importantes (ganando la competencia por gran margen), el trabajo de AlexNet también aporta resultados cualitativos.

Analizando los pesos resultantes, podemos ir entendiendo qué cosas capturan las diferentes partes de la red.



Figure 3: 96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2. See Section 6.1 for details.

Figure 3 shows the convolutional kernels learned by the network's two data-connected layers. The network has learned a variety of frequency- and orientation-selective kernels, as well as various colored blobs. Notice the specialization exhibited by the two GPUs, a result of the restricted connectivity described in Section 3.5. The kernels on GPU 1 are largely color-agnostic, while the kernels on GPU 2 are largely color-specific. This kind of specialization occurs during every run and is independent of any particular random weight initialization (modulo a renumbering of the GPUs).

El ganador en 2013 fue la ZFNet. En esencia es muy parecida a AlexNet pero con algunos hiperparámetros y decisiones diferentes se logró reducir el error de clasificación en casi 5%.

Más allá de la red en sí, el aporte de Zeiler y Fergus vino por el lado de la interpretación y el entendimiento visual de qué están aprendiendo las CNNs.

 Cornell University

Learn about arXiv becoming an independent nonprofit.

We gratefully acknowledge support from the Simons Foundation, member institutions, and all contributors. [Donate](#)

arXiv > cs > arXiv:1311.2901

Search... All fields Search

Help | Advanced Search

Computer Science > Computer Vision and Pattern Recognition

[Submitted on 12 Nov 2013 (v1), last revised 28 Nov 2013 (this version, v3)]

Visualizing and Understanding Convolutional Networks

Matthew D Zeiler, Rob Fergus

Large Convolutional Network models have recently demonstrated impressive classification performance on the ImageNet benchmark. However there is no clear understanding of why they perform so well, or how they might be improved. In this paper we address both issues. We introduce a novel visualization technique that gives insight into the function of intermediate feature layers and the operation of the classifier. We also perform an ablation study to discover the performance contribution from different model layers. This enables us to find model architectures that outperform Krizhevsky et al on the ImageNet classification benchmark. We show our ImageNet model generalizes well to other datasets: when the softmax classifier is retrained, it convincingly beats the current state-of-the-art results on Caltech-101 and Caltech-256 datasets.

Access Paper:
[View PDF](#)
[TeX Source](#)
[view license](#)

Current browse context:
cs.CV
[< prev](#) | [next >](#)
[new](#) | [recent](#) | [2013-11](#)
Change to browse by:
[cs](#)

VGG (usualmente VGG16 o VGG19) fue una red muy competitiva en la competencia del 2014.

El principal aporte con respecto a las anteriores fue mostrar que con todos filtros de 3×3 con stride 1, combinados de manera secuencial, se podía observar el mismo campo receptivo pero con más flexibilidad y menos parámetros.

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan* & Andrew Zisserman*

Visual Geometry Group, Department of Engineering Science, University of Oxford
{karen, az}@robots.ox.ac.uk

ABSTRACT

In this work we investigate the effect of the convolutional network depth on its accuracy in the large-scale image recognition setting. Our main contribution is a thorough evaluation of networks of increasing depth using an architecture with very small (3×3) convolution filters, which shows that a significant improvement on the prior-art configurations can be achieved by pushing the depth to 16–19 weight layers. These findings were the basis of our ImageNet Challenge 2014 submission, where our team secured the first and the second places in the localisation and classification tracks respectively. We also show that our representations generalise well to other datasets, where they achieve state-of-the-art results. We have made our two best-performing ConvNet models publicly available to facilitate further research on the use of deep visual representations in computer vision.

Con el cambio en los tamaños de los filtros consiguieron entrenar una red mucho más profunda (16 y 19 capas).

De todas maneras, seguía siendo una red bastante pesada para la época con casi 140 millones de parámetros (casi todos en las FC).

2.3 DISCUSSION

Our ConvNet configurations are quite different from the ones used in the top-performing entries of the ILSVRC-2012 (Krizhevsky et al., 2012) and ILSVRC-2013 competitions (Zeiler & Fergus, 2013; Sermanet et al., 2014). Rather than using relatively large receptive fields in the first conv. layers (e.g. 11×11 with stride 4 in (Krizhevsky et al., 2012), or 7×7 with stride 2 in (Zeiler & Fergus, 2013; Sermanet et al., 2014)), we use very small 3×3 receptive fields throughout the whole net, which are convolved with the input at every pixel (with stride 1). It is easy to see that a stack of two 3×3 conv. layers (without spatial pooling in between) has an effective receptive field of 5×5 ; three such layers have a 7×7 effective receptive field. So what have we gained by using, for instance, a stack of three 3×3 conv. layers instead of a single 7×7 layer? First, we incorporate three non-linear rectification layers instead of a single one, which makes the decision function more discriminative. Second, we decrease the number of parameters: assuming that both the input and the output of a three-layer 3×3 convolution stack has C channels, the stack is parametrised by $3(3^2C^2) = 27C^2$ weights; at the same time, a single 7×7 conv. layer would require $7^2C^2 = 49C^2$ parameters, i.e. 81% more. This can be seen as imposing a regularisation on the 7×7 conv. filters, forcing them to have a decomposition through the 3×3 filters (with non-linearity injected in between).

GoogLeNet fue la red ganadora del challenge de ImageNet 2014.

Uno de los aportes principales fue romper con la idea de “tengo este ladrillo (Convolutional Block), busquemos cómo apilar la mayor cantidad posible”.

Con la innovación de su *Inception Module* lograron entrenar una red de 22 capas pero que solamente tenía 5 millones de parámetros.

Going deeper with convolutions

Christian Szegedy

Google Inc.

Wei Liu

University of North Carolina, Chapel Hill

Yangqing Jia

Google Inc.

Pierre Sermanet

Google Inc.

Scott Reed

University of Michigan

Dragomir Anguelov

Google Inc.

Dumitru Erhan

Google Inc.

Vincent Vanhoucke

Google Inc.

Andrew Rabinovich

Google Inc.

Abstract

We propose a deep convolutional neural network architecture codenamed Inception, which was responsible for setting the new state of the art for classification and detection in the ImageNet Large-Scale Visual Recognition Challenge 2014 (ILSVRC14). The main hallmark of this architecture is the improved utilization of the computing resources inside the network. This was achieved by a carefully crafted design that allows for increasing the depth and width of the network while keeping the computational budget constant. To optimize quality, the architectural decisions were based on the Hebbian principle and the intuition of multi-scale processing. One particular incarnation used in our submission for ILSVRC14 is called GoogLeNet, a 22 layers deep network, the quality of which is assessed in the context of classification and detection.

GoogLeNet – Inception

La idea del Inception Module era dejar en la red la posibilidad de elegir el tamaño del filtro adecuado.

En vez de forzar nosotros el tamaño de la convolución, por qué no dejar que la red tenga la posibilidad de aprenderlo?

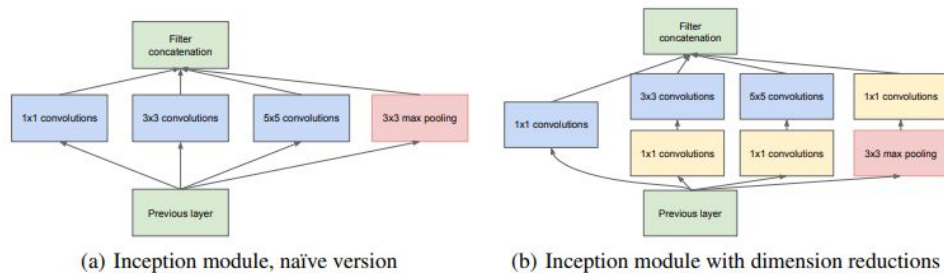
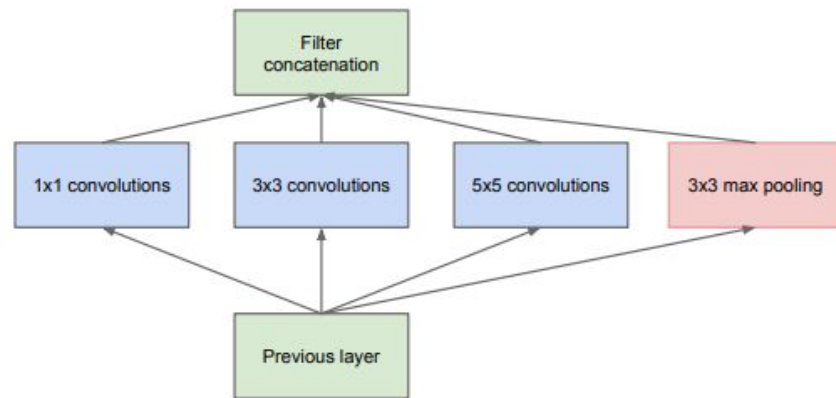


Figure 2: Inception module

GoogLeNet – Inception

En vez de pasar por una serie de convoluciones de forma serial, podemos hacer diferentes convoluciones en paralelo y luego simplemente concatenarlas en la nueva salida.

La idea es buena pero fácilmente se torna inmanejable computacionalmente al concatenar varios filtros.

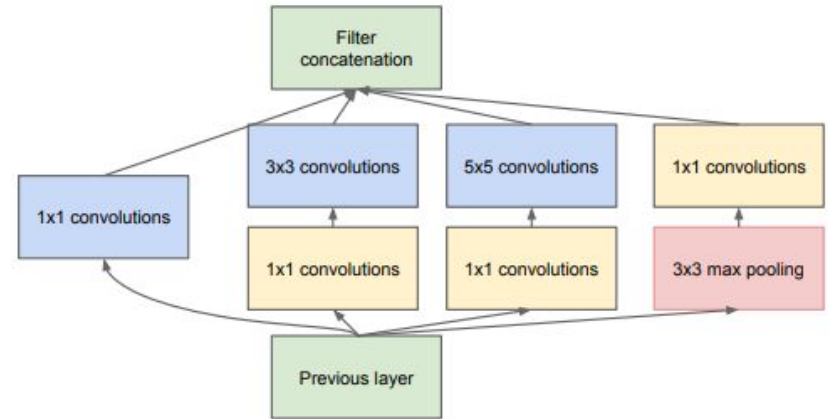


(a) Inception module, naïve version

GoogLeNet – Inception

Para que la cantidad de valores sea manejable, se aplican filtros de 1x1 que no cambian el tamaño de la imagen pero que reducen la cantidad de canales.

Funciona como una reducción de la dimensionalidad que la red puede aprender en cada punto.



(b) Inception module with dimension reductions

GoogLeNet – Global Average Pooling

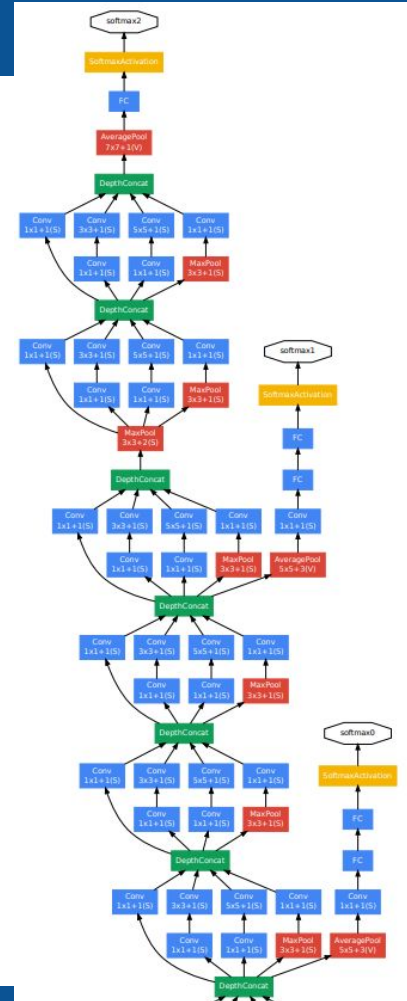
Otro de los aportes importantes fue el hecho de utilizar Global Average Pooling en vez de la capas de Flatten antes de llegar a la clasificación final.

En estas capas es donde estaban el 80% de los pesos de VGG y lo que la hacía tan pesada como red en general.

GoogLeNet – Profundidad

El uso de Global Average Pooling y los módulos de inception con el agregado de las convoluciones 1x1 hizo que la cantidad de parámetros en GoogLeNet fuera mucho menor a sus predecesores, lo que permitió apilar muchas más capas en profundidad.

Esto requirió de *clasificadores auxiliares* (*Deep supervision*) para evitar el problema de *vanishing gradients*.



El siguiente paso en el camino de ImageNet se dio en el 2015 mediante la arquitectura de ResNet.

El salto importante se dió al buscar soluciones ante el problema de no poder ahondar en la profundidad de las redes.

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun
Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

ResNet

“Is learning better networks as easy as stacking more layers?”

En el trabajo donde se presentan las ResNet lo primero que muestra el equipo es lo que sucede al entrenar dos CNNs muy similares de 20 y 56 capas.

Al poner más capas no solo no generaliza mejor, sino que ni siquiera aprende bien los datos de entrenamiento. El problema: los *vanishing gradients*.

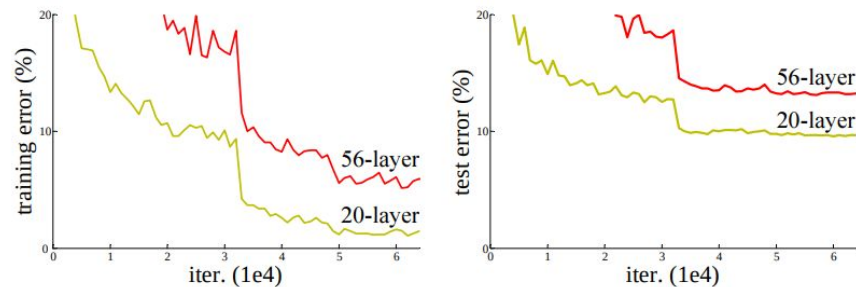


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

La solución: el Bloque Residual.

En una red *convencional* los inputs entran a un bloque, se transforman y salen.

En un Bloque Residual tenemos además un *skip connection*. El input entra a un bloque y se bifurca. Por un lado se transforma por las convoluciones normalmente. Por el otro se saltea las convoluciones y se suma al otro camino antes de hacer la activación final.

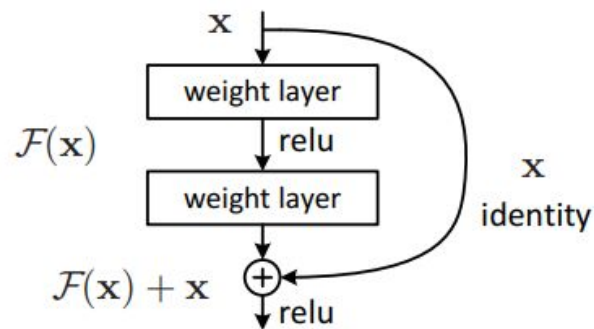


Figure 2. Residual learning: a building block.

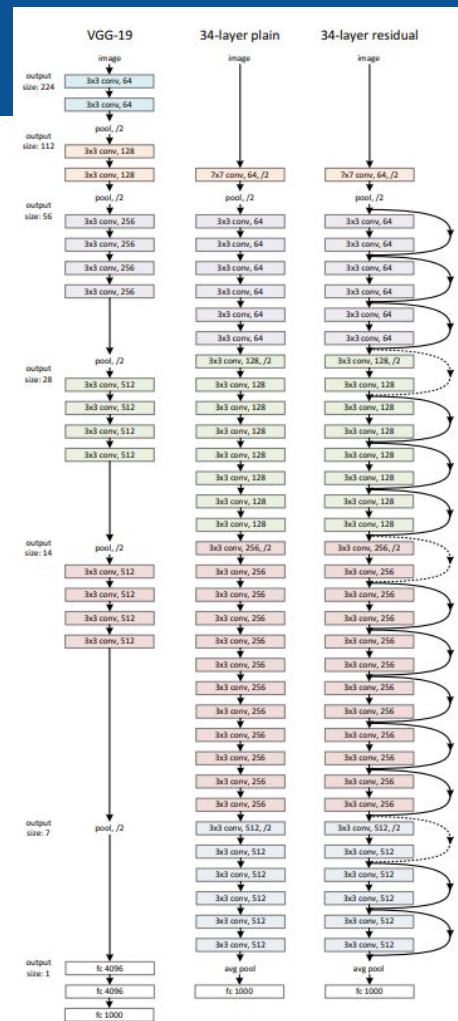
En las redes previas, las diferentes capas están buscando aproximar una función $H(x)$. En un Bloque Residual, se está sumando x directamente al final, por lo que ahora el bloque tiene que aprender una $F(x)$ tal que $H(x) = F(x) + x$.

Esto es conveniente por dos motivos:

- Nunca es malo agregar capas: es simple apagar todas las capas convolucionales y que la salida del bloque sea la identidad, por lo que es fácil que una red de 56 capas tenga mejor o igual rendimiento que una de 20 capas.
- El gradiente de una skip connection es 1 y al sumarse al gradiente del bloque mitiga el problema de los *vanishing gradients*.

En ImageNet 2015, la ganadora fue la ResNet152 consiguiendo un error total cercano al 3%.

Este error, vendido al público como “menor error que el humano”, fue el que determinó el fin del dataset y la competencia (aunque se hizo 2 años más).



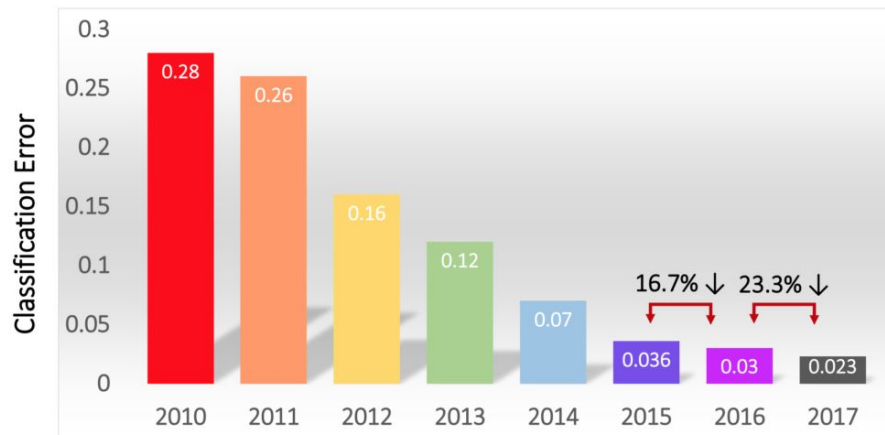
ResNext, Inception V2 y más

En los años subsiguientes, los ganadores fueron combinaciones de las ideas residuales y de inception.

Inception V3, Inception V4, DenseNet, ResNeXt.

Más allá de este dataset particular, los avances realizados abrieron el juego a un montón de otros problemas prácticos que hasta el 2012 resultaban casi imposible de atacar.

Classification Results (CLS)



Visualizing and Understanding CNNs

El trabajo de Zeiler y Fergus de 2013 es uno de los pilares para entender qué hacen las CNNs, qué es lo que aprenden en cada momento.

Esto sirvió para poder desarrollar mejores modelos específicos, pero también para entender qué partes de cada modelo se pueden reaprovechar en nuevos problemas.

Visualizing and Understanding Convolutional Networks

Matthew D. Zeiler

Dept. of Computer Science, Courant Institute, New York University

ZEILER@CS.NYU.EDU

Rob Fergus

Dept. of Computer Science, Courant Institute, New York University

FERGUS@CS.NYU.EDU

Visualizing and Understanding CNNs

Utilizan una *Deconvolutional Network* para ir del Feature Map a los píxeles originales.

Lo que hacen es aplicar capas *inversas* a las originales para poder estudiar ciertas activaciones particulares y entonces poder, mediante el mapeo inverso, ver qué píxeles influyeron en esa activación.

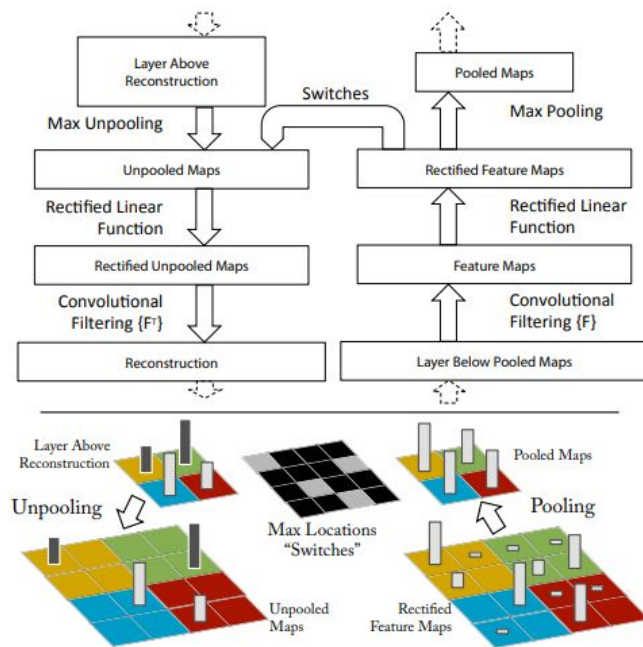
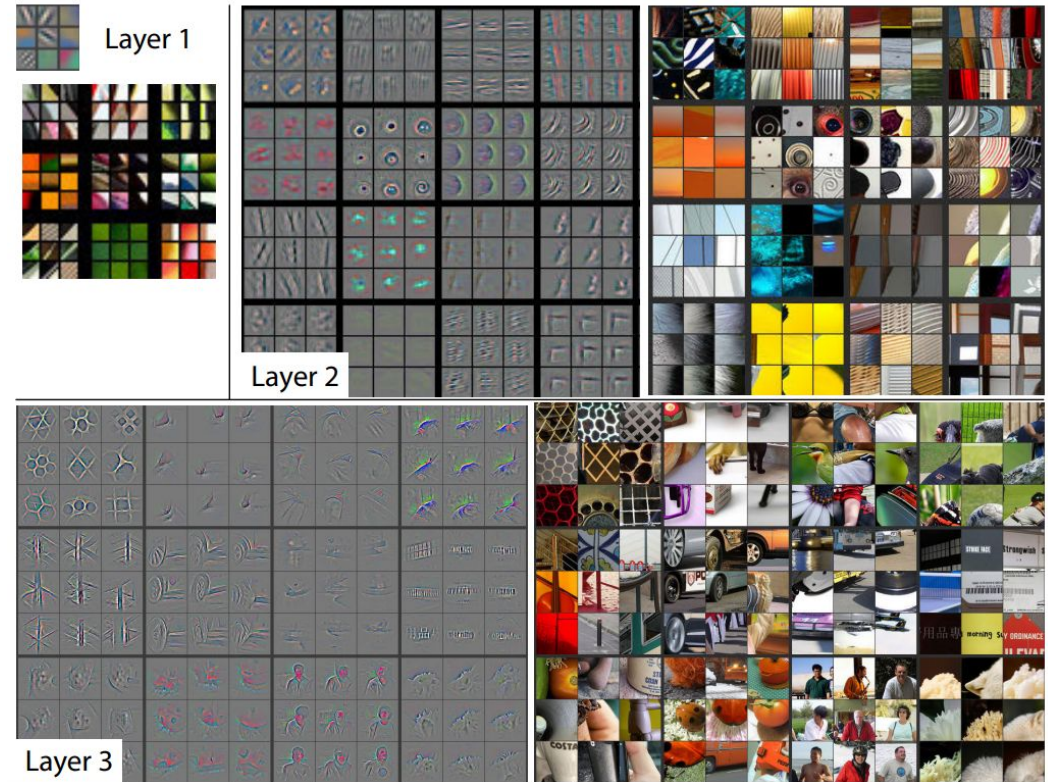


Figure 1. Top: A deconvnet layer (left) attached to a convnet layer (right). The deconvnet will reconstruct an approximate version of the convnet features from the layer beneath. Bottom: An illustration of the unpooling operation in the deconvnet, using *switches* which record the location of the local max in each pooling region (colored zones) during pooling in the convnet.

Visualizing and Understanding CNNs

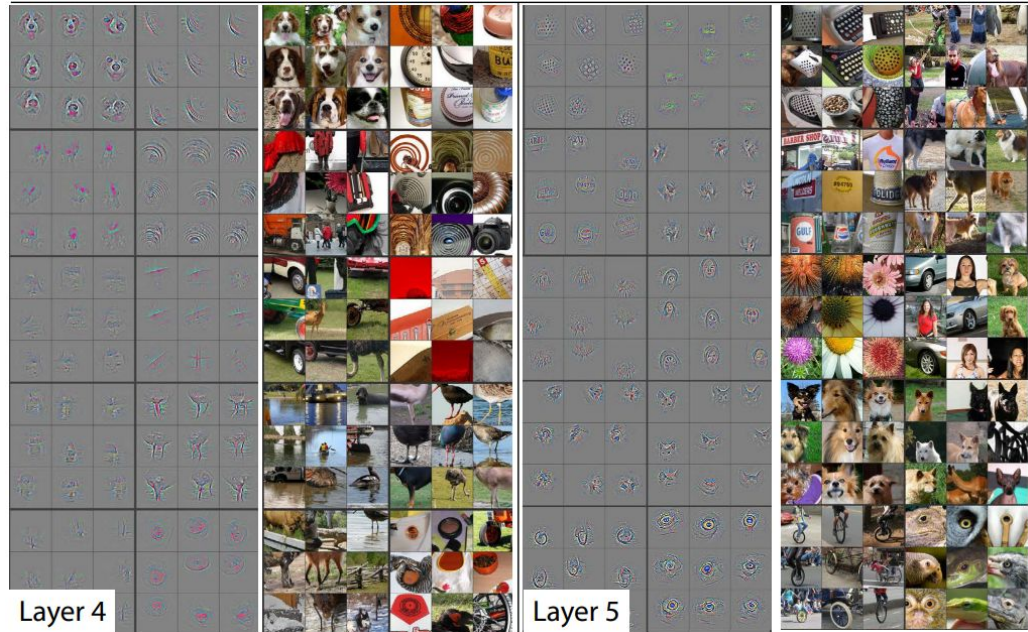
Usaron las Deconv Net para entender los diferentes filtros de las diferentes capas de redes entrenadas para ImageNet.

Podían ver cuáles eran las 9 imágenes que más activaban cada filtro, qué parte de la imagen lo activaba y cómo sería la reconstrucción de los píxeles que activó el filtro.



Visualizing and Understanding CNNs

Las primeras capas captaban patrones, formas y ángulos mucho más generales. Las últimas capas aprendían a mirar detalles mucho más refinados (caras, ruedas, etc).



Visualizing and Understanding CNNs

El último gran aporte del trabajo fue aplicar Transfer Learning de ImageNet a otros datasets.

El concepto de Transfer Learning ya existía en aprendizaje automático pero al entender qué aprendían las CNNs pudieron aplicarlo exitosamente.

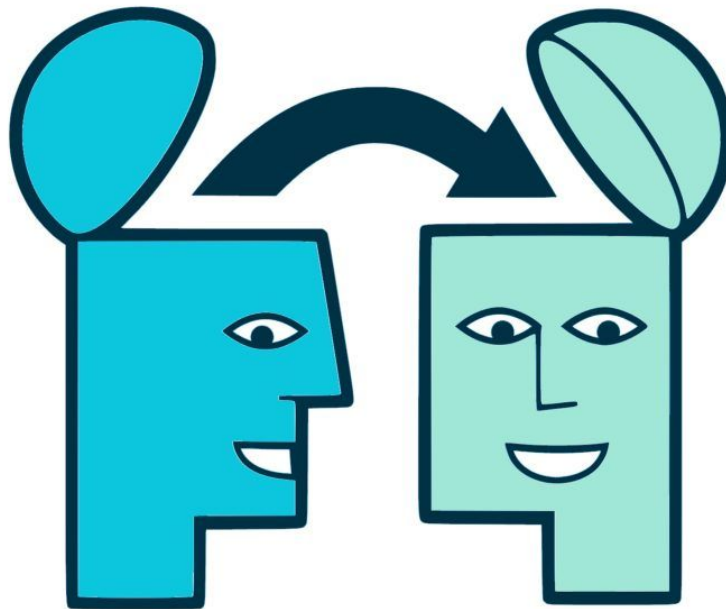
5.2. Feature Generalization

The experiments above show the importance of the convolutional part of our ImageNet model in obtaining state-of-the-art performance. This is supported by the visualizations of Fig. 2 which show the complex invariances learned in the convolutional layers. We now explore the ability of these feature extraction layers to generalize to other datasets, namely Caltech-101 (Fei-fei et al., 2006), Caltech-256 (Griffin et al., 2006) and PASCAL VOC 2012. To do this, we keep layers 1-7 of our ImageNet-trained model fixed and train a new

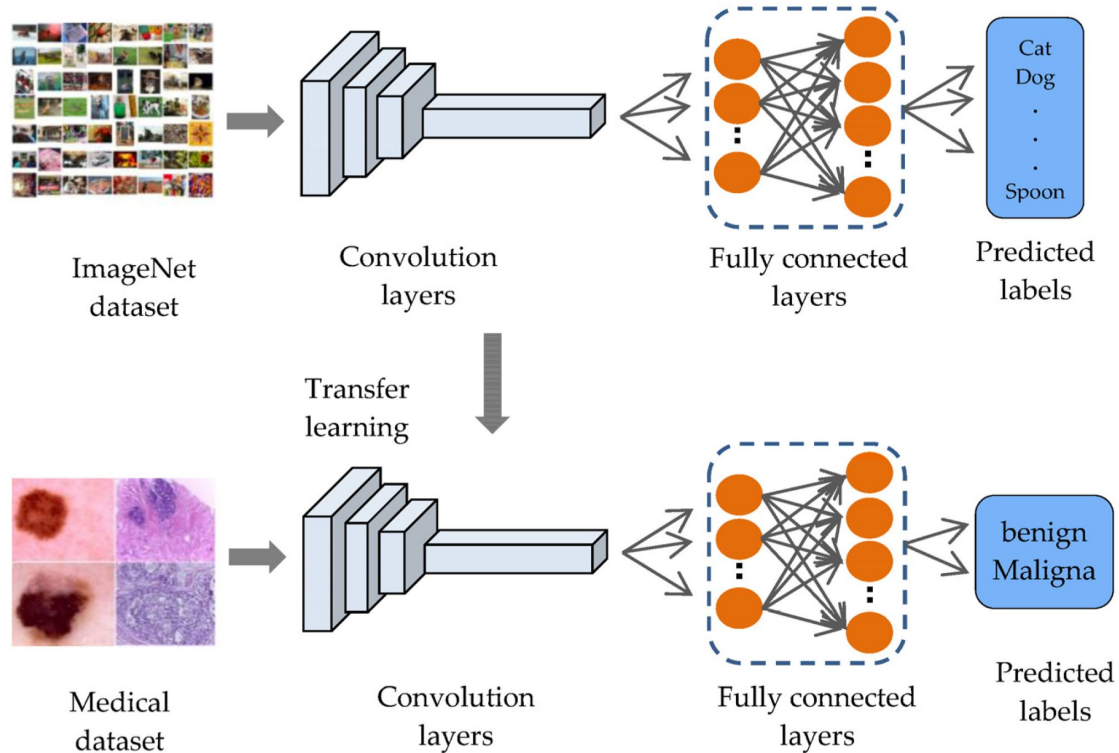
Transfer Learning

Empezar a entrenar desde 0 siempre es muy costoso.

Sabemos además que lo que aprenden las redes es mucho más general que simplemente un problema específico. La última capa *fully-connected* que devuelve la pertenencia a 1000 clases es muy dependiente de ImageNet-1K, pero todo lo anterior tiene información muy útil.



Transfer Learning



Transfer Learning – Chihuahuas

La clase pasada entrenamos una red *from scratch* para reconocer Chihuahuas y Muffins.

Comenzamos cerca del 80% en validación y en 10 épocas pudimos acercarnos al 90%.

Hicimos todo el trabajo de cero, ¿por qué no aprovechamos alguna red preentrenada?

Iniciando entrenamiento y validación...

Época 1/10

TRAIN -> Loss: 0.5482 | Acc: 71.32%

VAL -> Loss: 0.4213 | Acc: 79.09%

Época 2/10

TRAIN -> Loss: 0.3744 | Acc: 84.36%

VAL -> Loss: 0.3434 | Acc: 85.53%

Época 3/10

TRAIN -> Loss: 0.3021 | Acc: 87.51%

VAL -> Loss: 0.4128 | Acc: 82.89%

Época 4/10

TRAIN -> Loss: 0.2527 | Acc: 89.73%

VAL -> Loss: 0.2759 | Acc: 88.81%

Época 5/10

TRAIN -> Loss: 0.2177 | Acc: 91.10%

VAL -> Loss: 0.2760 | Acc: 88.17%

Época 6/10

TRAIN -> Loss: 0.1742 | Acc: 93.19%

VAL -> Loss: 0.2929 | Acc: 88.70%

Época 7/10

TRAIN -> Loss: 0.1498 | Acc: 94.16%

VAL -> Loss: 0.2914 | Acc: 88.81%

Época 8/10

TRAIN -> Loss: 0.1328 | Acc: 94.98%

VAL -> Loss: 0.2783 | Acc: 89.44%

Época 9/10

TRAIN -> Loss: 0.0875 | Acc: 96.72%

VAL -> Loss: 0.3221 | Acc: 89.02%

Época 10/10

TRAIN -> Loss: 0.0790 | Acc: 97.23%

VAL -> Loss: 0.2932 | Acc: 89.97%

Entrenamiento completado en 309.54 segundos.

Transfer Learning – Chihuahuas

En este caso, no definimos una red nosotros, si no que tomamos la ResNet18 como red base. Luego, tomamos la última capa (la *fully-connected*) y la reemplazamos por una nueva *fully-connected* que vaya a solo 2 clases.

A partir de ahí, solo vamos a entrenar esa última capa por un par de épocas.

```
norm_mean = [0.485, 0.456, 0.406]
norm_std = [0.229, 0.224, 0.225]

transformaciones = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(norm_mean, norm_std)
])

dataset_completo = datasets.ImageFolder(root='./chihuahua/train/', transform=transformaciones)

train_size = int(0.8 * len(dataset_completo))
val_size = len(dataset_completo) - train_size
train_db, val_db = torch.utils.data.random_split(dataset_completo, [train_size, val_size])

train_loader = DataLoader(train_db, batch_size=32, shuffle=True)
val_loader = DataLoader(val_db, batch_size=32, shuffle=False)

modelo_transfer = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

for param in modelo_transfer.parameters():
    param.requires_grad = False

num_features = modelo_transfer.fc.in_features
modelo_transfer.fc = nn.Linear(num_features, 2)

modelo_transfer = modelo_transfer.to(device)

criterio = nn.CrossEntropyLoss()
optimizador = optim.Adam(modelo_transfer.fc.parameters(), lr=0.001)
```


Transfer Learning – Chihuahuas

Con una red preentrenada como ResNet que sabe extraer un montón de features visuales de imágenes, en una sola época ya estamos muy cerca del 99%.

```
inicio = time.time()

for epoca in range(3):
    modelo_transfer.train()
    for imagenes, etiquetas in train_loader:
        imagenes, etiquetas = imagenes.to(device), etiquetas.to(device)
        optimizador.zero_grad()
        perdida = criterio(modelo_transfer(imagenes), etiquetas)
        perdida.backward()
        optimizador.step()

    modelo_transfer.eval()
    correctos_val = 0
    with torch.no_grad():
        for imagenes, etiquetas in val_loader:
            imagenes, etiquetas = imagenes.to(device), etiquetas.to(device)
            _, prediccion = torch.max(modelo_transfer(imagenes), 1)
            correctos_val += (prediccion == etiquetas).sum().item()

    print(f"Época {epoca+1} | Acc en Validación: {100 * correctos_val / val_size:.2f}%")

print(f"Transfer Learning listo en {time.time() - inicio:.2f} segundos.")
```

Época 1 | Acc en Validación: 98.94%
Época 2 | Acc en Validación: 99.16%
Época 3 | Acc en Validación: 99.05%
Transfer Learning listo en 100.38 segundos.

Transfer Learning – Perros vs Gatos

En la primera clase hicimos un clasificador de Perros y Gatos para desconfiar de las particiones de un A/B Testing que nos entregaron.

Muy fácilmente conseguimos separar las clases con más del 95% de accuracy.

Sin saberlo, estábamos haciendo Transfer Learning desde una Red Neuronal Convolutiva preentrenada.

Desde cero:

```
Using device: cuda
Epoch [1/10], Loss: 1.3696, Val Accuracy: 0.6617
Epoch [2/10], Loss: 0.5979, Val Accuracy: 0.6725
Epoch [3/10], Loss: 0.5560, Val Accuracy: 0.7192
Epoch [4/10], Loss: 0.5241, Val Accuracy: 0.7327
Epoch [5/10], Loss: 0.4913, Val Accuracy: 0.7524
Epoch [6/10], Loss: 0.4664, Val Accuracy: 0.7544
Epoch [7/10], Loss: 0.4480, Val Accuracy: 0.7781
Epoch [8/10], Loss: 0.4334, Val Accuracy: 0.7754
Epoch [9/10], Loss: 0.4087, Val Accuracy: 0.7727
Epoch [10/10], Loss: 0.3977, Val Accuracy: 0.7652
```

Transfer Learning (ResNet18):

```
Using device: cuda
Epoch [1/5], Loss: 0.1979, Val Accuracy: 0.9675
Epoch [2/5], Loss: 0.0950, Val Accuracy: 0.9716
Epoch [3/5], Loss: 0.0703, Val Accuracy: 0.9783
Epoch [4/5], Loss: 0.0663, Val Accuracy: 0.9770
Epoch [5/5], Loss: 0.0541, Val Accuracy: 0.9770
```